

Embeddings y similitud semántica

MMIA 6013 — IA Generativa y Agentes · Sesión 06 · lunes 21 de septiembre de 2026

Semana 2 · RA-2 · de los 110 minutos, 60 son actividad y 50 son este deck

Daniel Andrés Riofrío Almeida

Recuperar, no ajustar: la semana empieza aquí

- La tesis de la semana: «el modelo no conoce mis documentos» no se arregla ajustando el modelo, se arregla recuperando
- Recuperar exige comparar textos por significado, no por palabras compartidas
- Hoy definimos la unidad que se compara: un vector por texto
- Martes, el índice. Miércoles, fragmentar y recuperar. Jueves, medir el orden
- Sábado, Taller 2 — sistema RAG sobre base vectorial, 25 %
- Hoy se nombra RAG; se enseña el miércoles. Hoy se enseña el vector

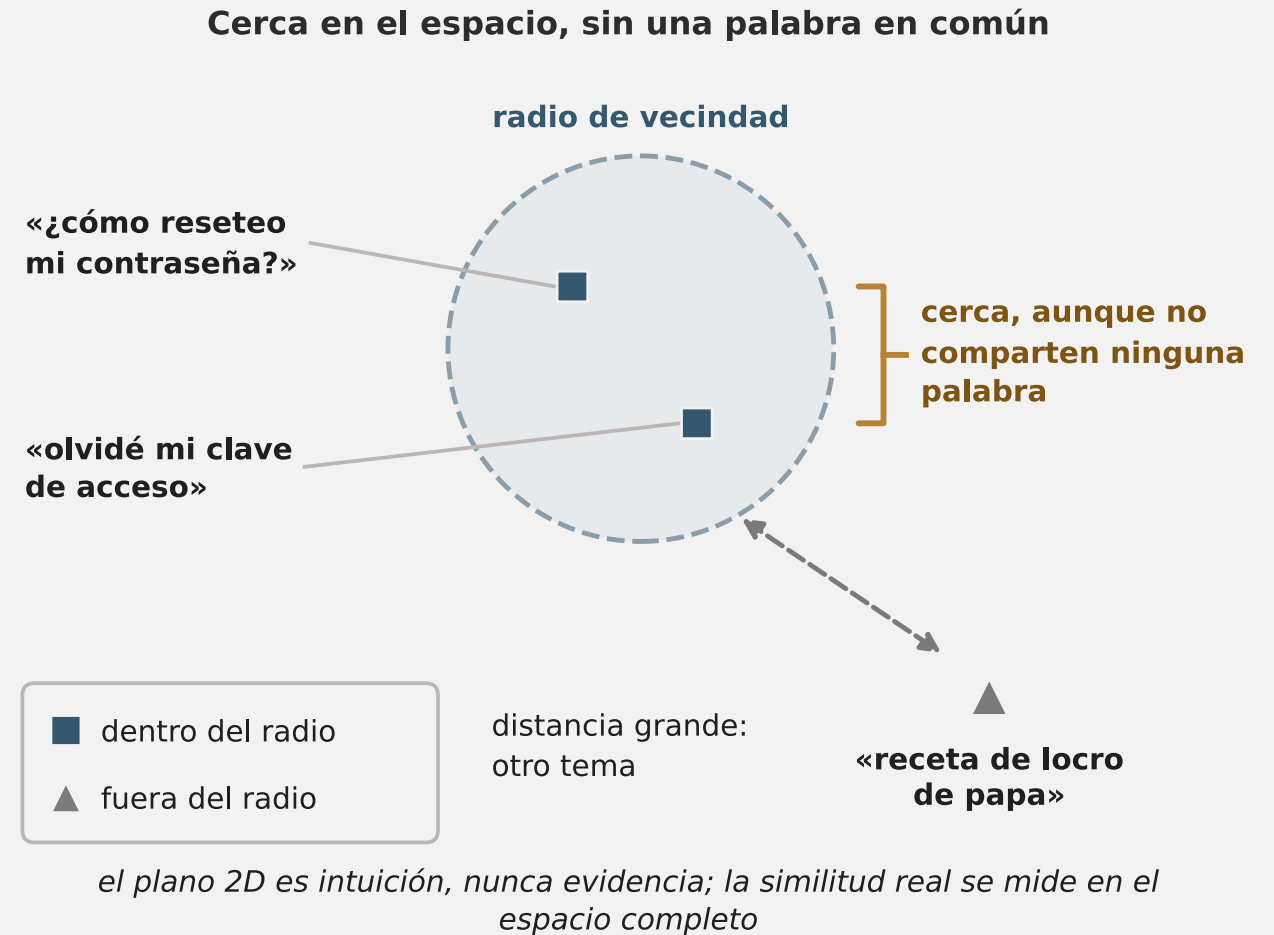
Fuente: slides_v2/sesion-05/sesion-05.qmd, diapositiva de cierre «El lunes: recuperar, no ajustar», desactivada el 2026-09-18 y conservada comentada al final del archivo (la tesis es la misma; como ese cierre no se proyectó, hoy se enuncia en vez de recordarse) y slides_v2/sesion-05/contenido-base.md §13.3 (la promesa explícita de no definir hoy el vocabulario de la semana); curso/planificacion/plan.md, filas 06 a 10 (temas de la semana 2 y Taller 2 del 2026-09-26, 25 %); proceso/contexto/glosario.md (Embedding se introduce en 06, Vector database en 07, Chunking, Retrieval y RAG en 08)

Qué es un embedding

06

Un vector con una promesa

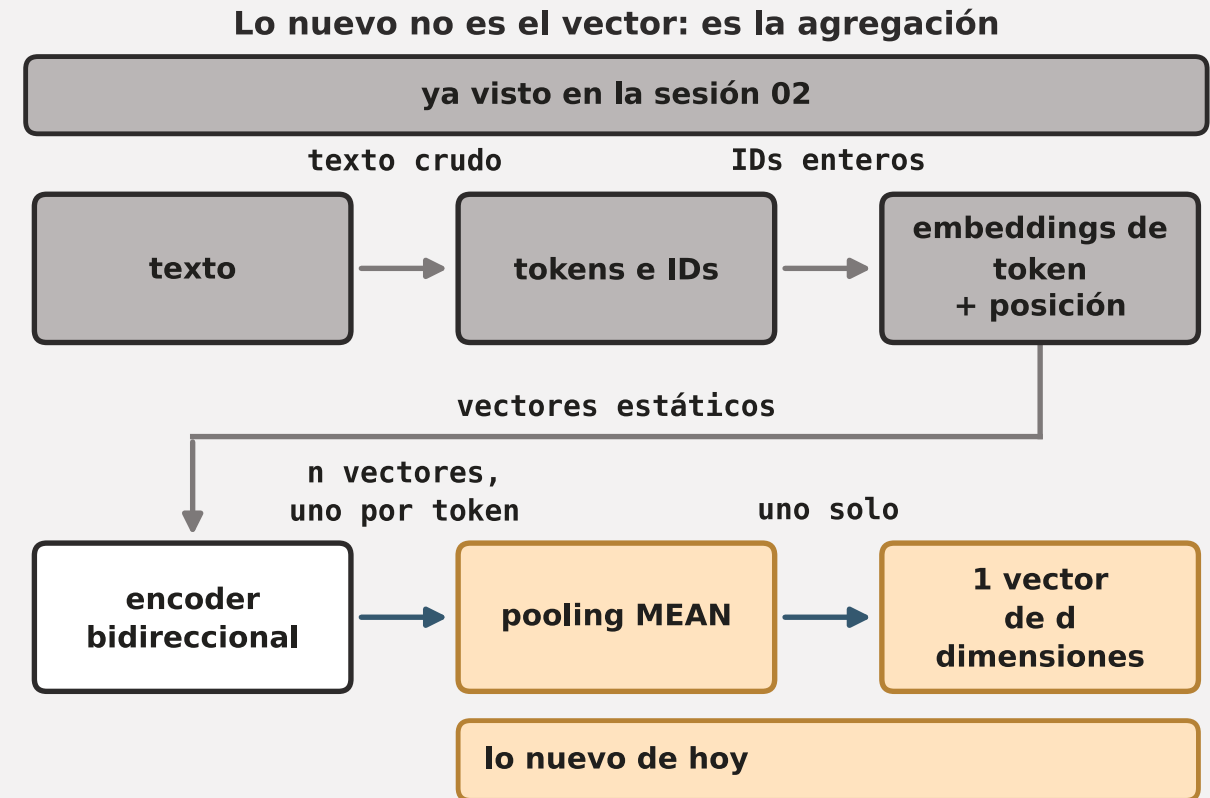
- Un embedding es una función $f : \text{texto} \rightarrow \mathbb{R}^d$, con $d = 384, 1024, 1536$ o 3072
- La promesa: textos semánticamente similares quedan cerca en el espacio
- «El perro corre en el parque» $\approx$ «El can galopa en el jardín»; $\neq$ «La computadora calcula π »
- ¿Similar en qué? Lo fija el entrenamiento: con reseñas etiquetadas, agrupa por sentimiento
- Esa cercanía se entrena. No sale gratis por tener vectores
- Y se enuncia respecto de UNA medida de cercanía: SBERT la enuncia con coseno



Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §2.1, líneas 30–35 (la definición y los dos ejemplos son literales); Reimers & Gurevych (2019), Sentence-BERT, §1, p. 1, nota al pie 2 (“semantically similar sentences are close in vector space”) y §4, p. 4 (“we always use cosine-similarity to compare the similarity between two sentence embeddings”); los cuatro valores de d son las filas de [fuentes/modelos/modelos-2026-1.json](#), clave `embeddings`, verificadas el 2026-08-27, y no se escriben de memoria; Alammari & Grootendorst (2024), cap. 10, §«Embedding Models», p. 291 y Fig. 10-3 (qué cuenta como «similar» lo fija el entrenamiento, no el método)

De n vectores de token a un vector de texto

- Sesión 02: la capa de embedding da siempre el mismo vector al mismo ID de token
- El encoder bidireccional los vuelve contextualizados: «banco de peces» ≠ «banco de crédito»
- Pero devuelve n vectores, uno por token, y la búsqueda necesita uno
- SBERT añade una operación de pooling sobre la salida: CLS, MEAN o MAX
- MEAN es el defecto y está medido: 87,44 frente a 69,92 de MAX en STSb
- El de texto —oración, párrafo o documento entero— es el único que se indexa; no es el de token

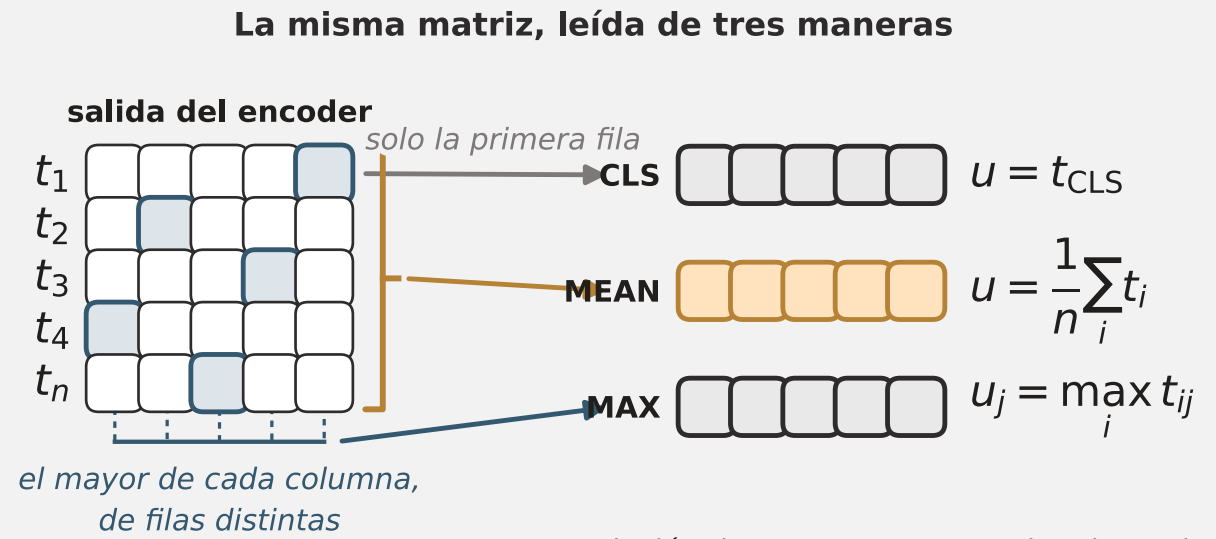


en la Tabla 6 del paper (dev de STSb) CLS queda a menos de un punto de MEAN; el pooling que se desploma es MAX (Reimers y Gurevych 2019)

Fuente: Reimers & Gurevych (2019), §3, p. 3 (“SBERT adds a pooling operation to the output of BERT / RoBERTa to derive a fixed sized sentence embedding”; MEAN es la configuración por defecto) y §6, Tabla 6, p. 7 (ablación de pooling sobre el dev set de STSb: MEAN 87.44, CLS 86.62, MAX 69.92); slides_v2/sesion-02/contenido-base.md §4.1 (Raschka 2024, §2.8, p. 45: la capa de embedding mapea siempre el mismo ID al mismo vector) y §10.3 (encoder = atención bidireccional); el ejemplo de «banco» es de curso/actividades/semana-2/s2-lun-explicacion.html, líneas 53–58; Alammari & Grootendorst (2024), cap. 2, §«Token Embeddings» pp. 57–58, §«Creating Contextualized Word Embeddings» pp. 58–61 y §«Text Embeddings (for Sentences and Whole Documents)» pp. 61–63 (los tres embeddings, con esos nombres, y el promedio como método por defecto)

Las tres formas de agregar, y qué le pasa a cada una

- El encoder devuelve $t_1, \dots, t_n$: un vector de d dimensiones por token, ya contextualizado
- MEAN promedia los n : $u = \frac{1}{n} \sum_{i=1}^n t_i$. Cada token pesa igual y ninguno decide solo
- CLS toma uno: $u = t_{\text{CLS}}$. Es el token que BERT entrenó para clasificar, no para comparar
- MAX va dimensión por dimensión: $u_j = \max_i t_{ij}$. Se queda el pico de cada una, de tokens distintos
- Las tres salen del MISMO encoder y dan el mismo d : lo que cambia es qué información sobrevive
- Con el objetivo de regresión, MAX pierde 17,5 puntos contra MEAN; con el de clasificación, 1,7



estrategia	correlación de Spearman ×100, dev de STSb	
	objetivo de clasificación (NLI)	objetivo de regresión (STSb)
MEAN	80,78	87,44
CLS	79,80	86,62
MAX	79,07	69,92

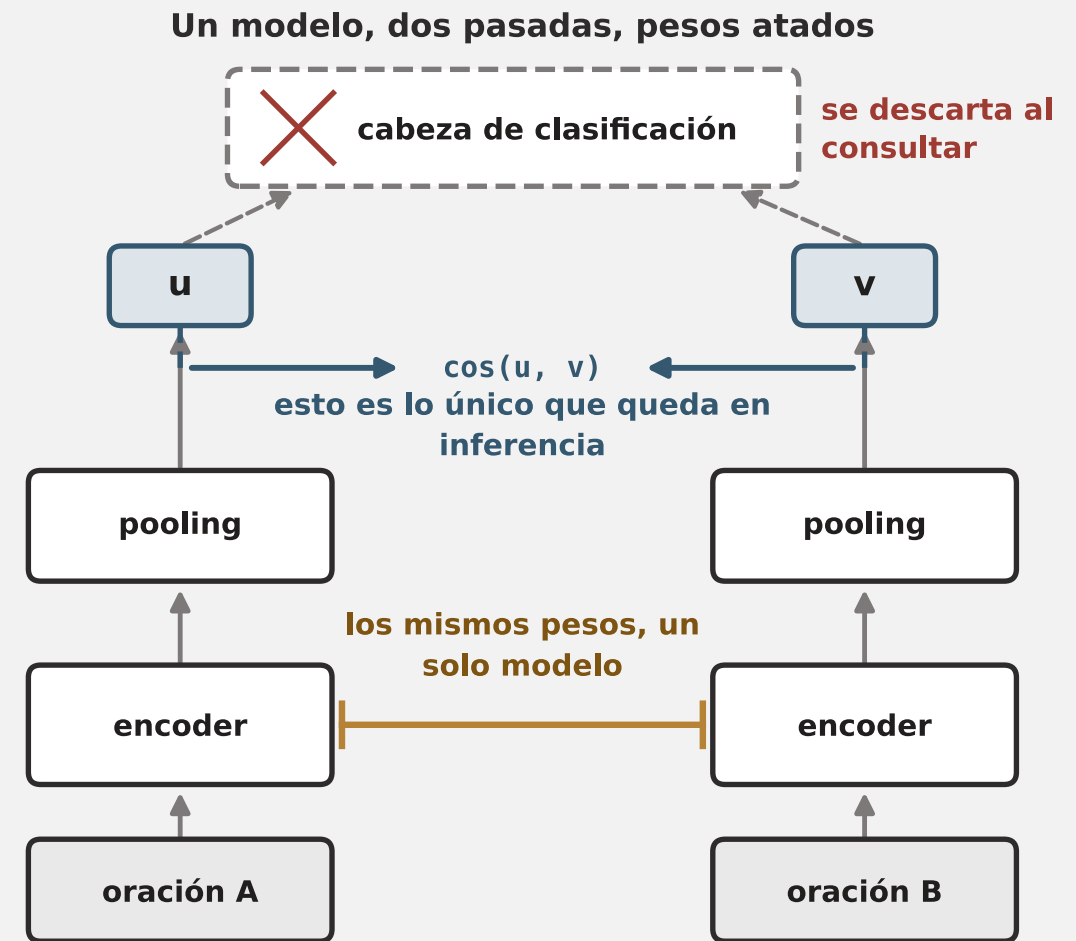
Fuente: Reimers & Gurevych (2019), §3, p. 3, literal: “We experiment with three pooling strategies: Using the output of the CLS-token, computing the mean of all output vectors (MEAN-strategy), and computing a max-over-time of the output vectors (MAX-strategy). The default configuration is MEAN”; §6, Tabla 6, p. 7 (la ablación, correlación de Spearman ×100 sobre el dev set de STSb: MEAN 80,78 y 87,44; CLS 79,80 y 86,62; MAX 79,07 y 69,92) y §6, p. 7 (“the MAX strategy performs significantly worse than MEAN or CLS-token strategy” con el objetivo de regresión, y “the pooling strategy has a rather minor impact” con el de clasificación); slides_v2/sesion-06/contenido-base.md §4; fuentes/modelos/modelos-2026-1.json, fila embed_notebook_s2 (pooling mean, verificado 2026-09-21 contra la configuración del propio modelo). Las tres fórmulas son la escritura en notación del curso de esas tres frases: [AGREGADO - no está en las fuentes], que no dan ecuación

Cómo se entrena

06

Una red siamesa es un modelo aplicado dos veces, no dos modelos

- Dos pasadas del mismo modelo, con pesos atados; cada rama produce su vector
- El objetivo se calcula sobre el par, no sobre cada texto por separado
- Tres funciones objetivo, no una: clasificación, regresión y triplet
- El resultado principal es clasificación sobre NLI: 570 000 + 430 000 pares, una época
- Las tres son contrastivas: el par —o el triple— es la unidad, no el texto suelto
- En inferencia solo quedan u , v y el coseno: la cabeza de clasificación se descarta

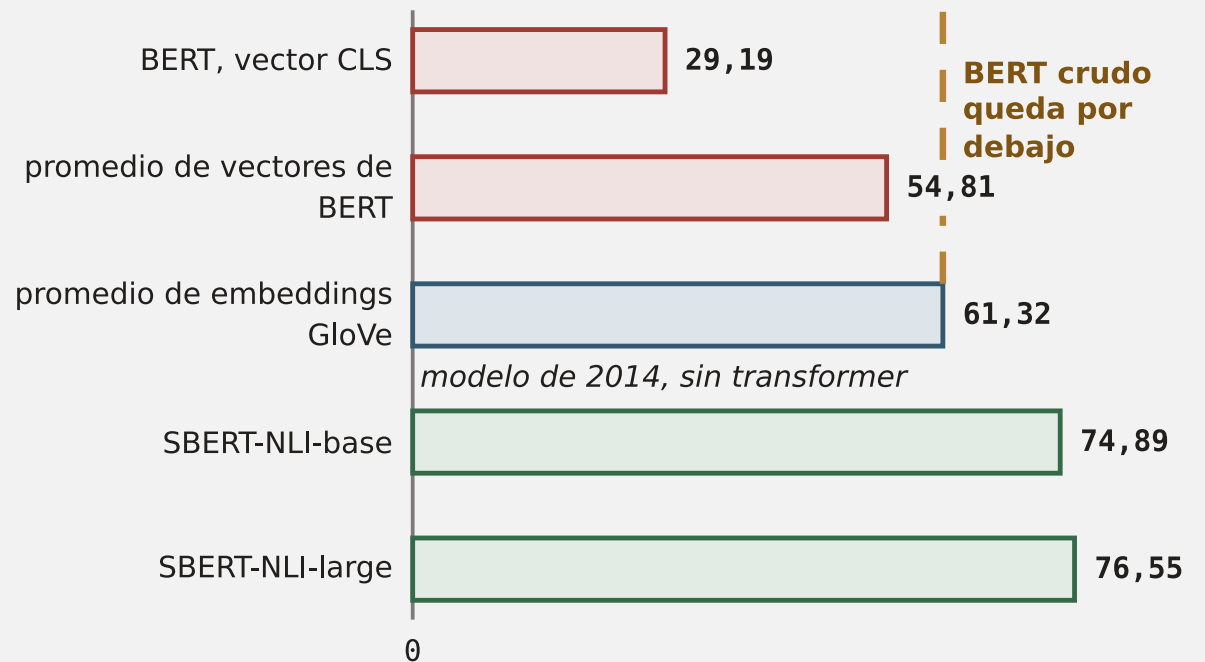


Fuente: Reimers & Gurevych (2019), §3 y Fig. 1, p. 3 (estructura siamesa con pesos atados; las tres funciones objetivo: softmax sobre $(u, v, |u-v|)$, regresión sobre $\cosine\text{-sim}(u,v)$ con MSE, y triplet con distancia euclídea y $\epsilon = 1$), §3.1, p. 3 (SNLI 570 000 pares + MultiNLI 430 000, una época, lote 16, Adam $2e-5$, pooling MEAN) y §6, p. 7 (“At inference... only the sentence embeddings u and v are used in combination with cosine-similarity”); fuentes/notas-teoricas/s2-rag-y-vector-search.md, §2.2, líneas 39–43, y Alammari & Grootendorst (2024), cap. 10, §«What Is Contrastive Learning?», p. 291 y §«SBERT», p. 295 (el bi-encoder entero es aprendizaje contrastivo; los pares de NLI son los ejemplos contrastivos, §«Generating Contrastive Examples», p. 296)

BERT crudo no sirve como buscador

- Correlación de Spearman con etiquetas humanas, promedio de siete tareas STS, con coseno
- BERT CLS 29,19 · promedio de BERT 54,81 · GloVe 61,32 · SBERT-NLI base 74,89 y large 76,55
- El paper lo dice así: often worse than averaging GloVe embeddings
- Y los mismos vectores rinden 84,94 en SentEval, donde se entrena un clasificador encima
- El coseno trata todas las dimensiones por igual; un clasificador aprende a ponderarlas
- No es que el modelo sea peor: nadie le enseñó qué dimensiones importan

Con coseno decide el entrenamiento, no el tamaño
correlación de Spearman $\rho \times 100$ · promedio de siete tareas STS · coseno · Reimers y Gurevych 2019, Tabla 1



cifras de 2019 sobre los modelos que ese trabajo midió; los mismos vectores rinden 84,94 en SentEval, donde se entrena un clasificador encima

Fuente: Reimers & Gurevych (2019), Tabla 1, p. 4 (columna Avg. de siete tareas STS, sin entrenamiento específico de STS), §1, p. 1 (“this common practice yields rather bad sentence embeddings, often worse than averaging GloVe embeddings”), Tabla 5 y §5, p. 7 (SentEval: promedio de BERT 84,94 · GloVe 81,52 · SBERT-NLI-base 87,41; “Cosine-similarity treats all dimensions equally... SentEval fits a logistic regression classifier”). La misma conclusión, sin cifras y cinco años después, en Alammari & Grootendorst (2024), cap. 10, §«SBERT», p. 294, y cap. 2, §«Text Embeddings», p. 62. Cifras de 2019, sobre los modelos que el paper midió; no son una afirmación sobre modelos actuales

Bi-encoder y cross-encoder: 65 horas o 5 segundos

- Cross-encoder: el par entra junto y sale un puntaje. Más preciso, y no produce vectores
- Comparar 10 000 oraciones entre sí son 49 995 000 inferencias: ~65 h en una V100
- Bi-encoder: cada texto se embebe por separado y una sola vez, ~5 s
- Comparar después los vectores por coseno cuesta ~0,01 s
- Se paga: ~7 puntos de Spearman menos en la evaluación entre temas del corpus AFS
- Precalcular es lo que hace posible buscar. El cross-encoder vuelve el miércoles

Fuente: Reimers & Gurevych (2019), §1, pp. 1-2 (BERT como cross-encoder; $n \cdot (n-1) / 2 = 49\,995\,000$ inferencias y unas 65 horas en una GPU V100 para $n = 10\,000$), abstract y §8, p. 8 (~5 segundos para calcular los vectores y ~0.01 s para compararlos), §4.3 y Tabla 3, p. 5 (corpus AFS, evaluación entre temas: base $57.23 \rightarrow 50.65$, large $60.34 \rightarrow 53.10$). El término bi-encoder no aparece en Reimers & Gurevych (2019), que dice “siamese network structure”; si lo usa Alammari & Grootendorst (2024), cap. 10, §«SBERT», p. 295 (“also referred to as a bi-encoder or SBERT”), que además precisa que un cross-encoder no genera embeddings (p. 294). Registrado en proceso/contexto/glosario.md

Apuesta antes de medir — 10 minutos, sin computadora

- En grupo, ordenen estos cuatro pares de MÁS a MENOS similares. Escriban su orden
- 1. «el gato duerme en el sofá» ↔ «un felino descansa en el mueble»
- 2. «el gato duerme en el sofá» ↔ «el gato no duerme en el sofá»
- 3. «el gato duerme en el sofá» ↔ «python lanza una excepción»
- 4. «¿cómo facturo con RUC?» ↔ «pasos para emitir factura electrónica»
- El orden escrito antes de medir es el que hace que el resultado enseñe algo

Coseno, producto punto y qué cambia la
normalización

06

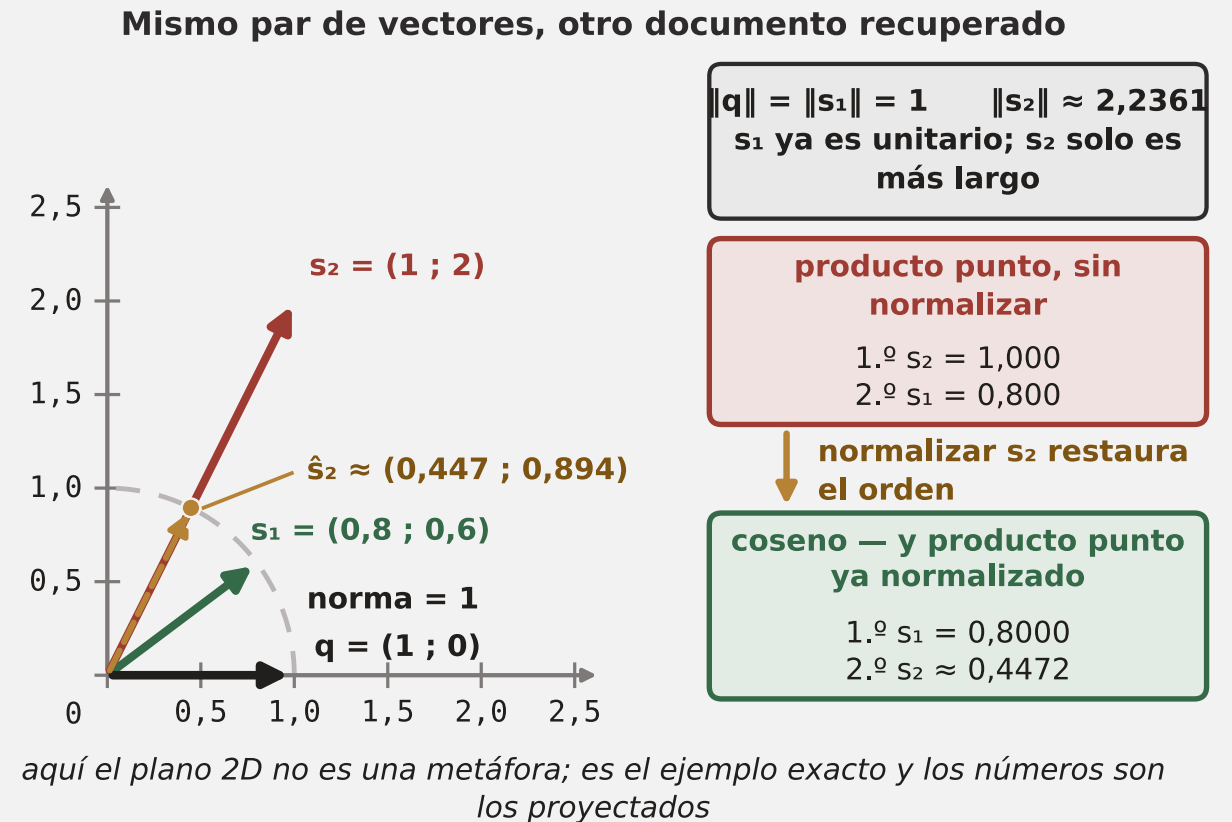
Tres medidas, y cuándo son la misma

- Coseno: $\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$, en $[-1, 1]$. No depende de la magnitud
- Producto punto: $u \cdot v$, sin cota, y más barato — sin raíces ni divisiones
- Euclídea: $\|u - v\|$, en $[0, \infty)$
- Si $\|u\| = \|v\| = 1$: $\cos(u, v) = u \cdot v$ exactamente, y $\|u - v\|^2 = 2 - 2 \cos(u, v)$
- Normalizados, los tres producen el MISMO ranking: medido, ρ de Spearman idéntica en las tres
- Por eso los índices normalizan al insertar, y la elección de métrica deja de importar

Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §2.3, líneas 55–64 (las tres fórmulas y sus rangos; “equivalente a coseno si los vectores están normalizados. Más eficiente”); [curso/actividades/semana-2/s2-lun-explicacion.html](#), línea 66 (con vectores normalizados la euclídea ordena igual que el coseno) y línea 113 (“si $\|a\|=\|b\|=1$, esa división es entre 1... los índices normalizan una vez al insertar”); Reimers & Gurevych (2019), §4, p. 4 (midieron Manhattan y euclídea negativas: “the results for all approaches remained roughly the same”); [fuentes/modelos/modelos-2026-1.json](#), fila `embed_api_economico`, verificada 2026-08-27 (la documentación oficial declara salida normalizada a longitud 1 “de modo que coseno y distancia euclídea producen el mismo orden”). Alammari & Grootendorst (2024), cap. 10, §«Supervised», p. 310: sobre el mismo conjunto de validación, un modelo de embeddings ya ajustado da ρ de Spearman idéntica con coseno, euclídea y producto punto, y Pearson distintas — el orden coincide, el valor no. El desarrollo algebraico de $\|u - v\|^2 = 2 - 2 \cos(u, v)$ es [AGREGADO - no está en las fuentes]: es la expansión del cuadrado del binomio; la conclusión de que ordenan igual sí es literal de la fuente

Sin normalizar, no cambia el margen: cambia el ganador

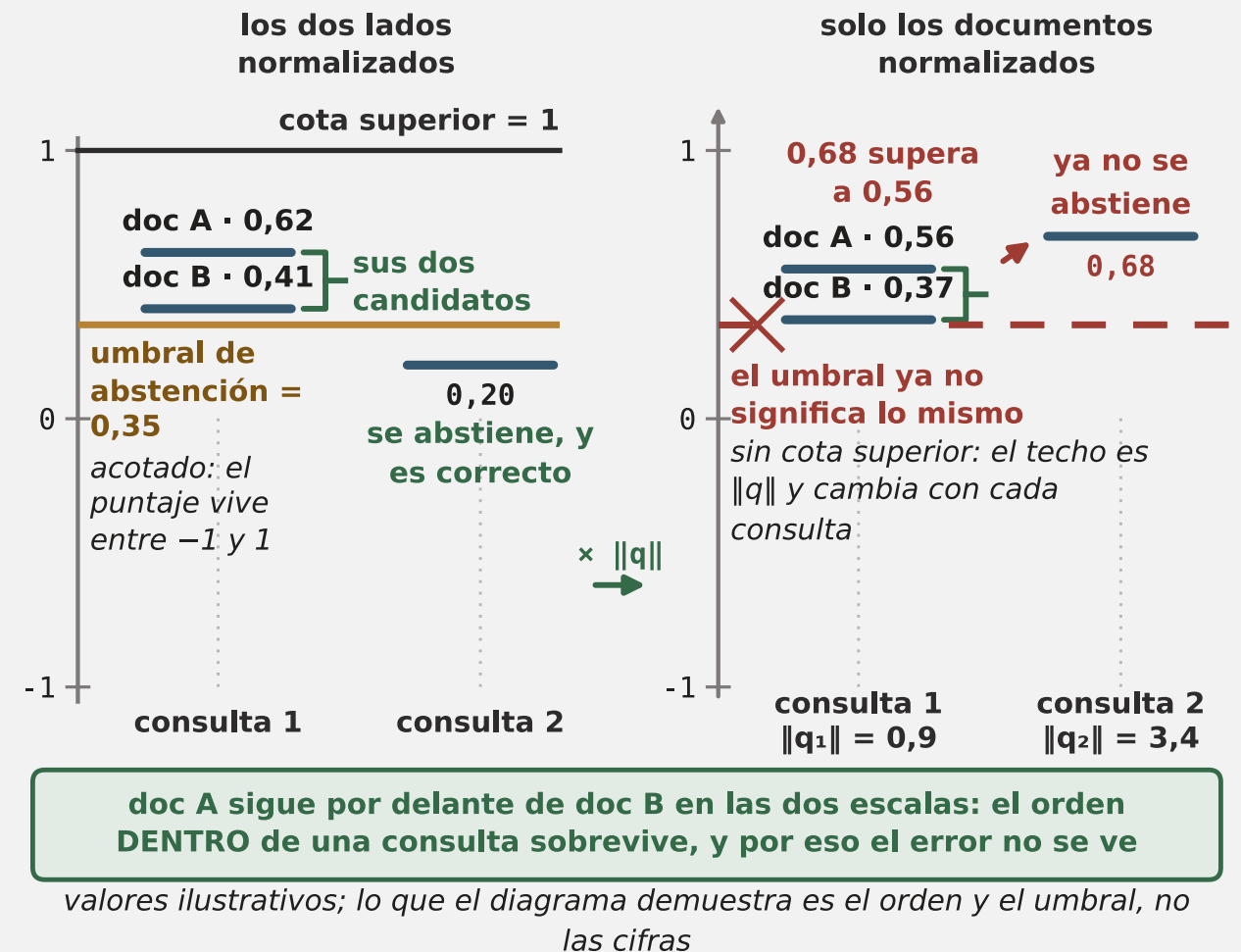
- Consulta $q = (1; 0)$ y dos candidatos: $s_1 = (0,8; 0,6)$ con $\|s_1\| = 1$, y $s_2 = (1; 2)$ con $\|s_2\| = \sqrt{5} \approx 2,236$
- Producto punto: $q \cdot s_1 = 0,80$ y $q \cdot s_2 = 1,00 \rightarrow$ gana s_2
- Coseno: $\cos(q, s_1) = 0,80$ y $\cos(q, s_2) = \frac{1}{\sqrt{5}} \approx 0,447 \rightarrow$ gana s_1
- El orden se invierte. Es el mismo par de vectores y otro documento recuperado
- Normalizamos s_2 : $\hat{s}_2 \approx (0,447; 0,894)$, y el orden vuelve al del coseno porque $q \cdot \hat{s}_2 \approx 0,447$
- s_2 no gana por parecerse más a la consulta: gana por ser más largo



Fuente: el contraejemplo en $\mathbb{R}^2$ es [AGREGADO - no está en las fuentes]: construido para este deck con aritmética verificable a mano, porque ninguna fuente del curso trae uno y el objetivo del plan para la sesión 06 exige demostrarlo, no afirmarlo. Corroborado por Alammar & Grootendorst (2024), cap. 10: sobre el mismo conjunto de validación y con vectores sin normalizar, la correlación de Spearman con producto punto queda por debajo de la de coseno las tres veces que el libro la mide $-0,3701$ vs. $0,6027$ (p. 300), $0,6624$ vs. $0,7251$ (p. 304) y $0,7345$ vs. $0,8121$ (p. 307)— y coincide exactamente con ella cuando los vectores tienen norma 1 (p. 310). Atribuir la brecha a la magnitud es inferencia de este deck: el libro publica los números y no los explica. La Parte 3 del notebook fija `normalize_embeddings=True` (curso/actividades/_fuentes/s2-lun.py, línea 36); desde el 2026-09-19 el ejercicio 5.2 (líneas 191–207) repite la búsqueda con `normalize_embeddings=False` y compara los dos rankings. Las definiciones de las dos medidas son de fuentes/notas-teoricas/s2-rag-y-vector-search.md, §2.3, líneas 55–64; la distinción está registrada en proceso/contexto/glosario.md como «Similitud coseno vs. producto punto sin normalizar»

Normalizar un solo lado: el orden intacto y el umbral roto

- Documentos normalizados y consulta no:
 $q \cdot \hat{s}_i = \|q\| \cos(q, s_i)$
- $\|q\|$ es una constante positiva de esa consulta: multiplica a todos los candidatos igual
- El ranking de esa consulta es idéntico al del coseno. Nada parece fallar
- Pero el puntaje sale de $[-1, 1]$ y deja de ser comparable entre consultas
- «Abstente si el mejor puntaje $< 0,35$ » ya no significa lo mismo dos veces
- El Taller 2 mide dos tasas de abstención, correcta e indebida: ese umbral es la Parte 2.b



Fuente: [AGREGADO - no está en las fuentes]. La derivación $q \cdot \hat{s} = \|q\| \cdot (\hat{q} \cdot \hat{s}) = \|q\| \cdot \cos(q, s)$ es de este deck y es aritmética verificable; proceso/contexto/glosario.md la registra como distinción vigilada «Normalizar los documentos vs. normalizar los dos lados». Las dos premisas sí están citadas, en la revisión del 2026-09-19 del enunciado: curso/talleres/taller-02-rag-vector-search.md, §«Parte 2.a» (golden set de diez pares con al menos dos preguntas negativas; líneas 181-197) y §«Parte 2.b» (las dos tasas de abstención, “la correcta (negativas en las que el sistema se abstuvo) y la indebida (respondibles en las que también se abstuvo)”, y Hit Rate y MRR solo sobre las respondibles; líneas 199-211). El vínculo entre umbral y abstención es propuesta de este deck

Dos líneas deciden si esto es un coseno

- Son las dos líneas del notebook de hoy, y son inseparables
- El producto punto de abajo solo es un coseno porque la línea de arriba normalizó
- La normalización no ocurre sola: es un argumento que alguien escribió

```
# 1) al embeber (s2-lun.py, línea 36)
return np.asarray(
    _modelo.encode(textos, normalize_embeddings=True))

# 2) al buscar (s2-lun.py, línea 129)
scores = V_corpus @ V_consultas[idx_consulta]
# vectores normalizados -> dot = coseno
```

Fuente: `curso/actividades/_fuentes/s2-lun.py`, líneas 36 y 129 (fuente canónica de los dos notebooks de la sesión); el comentario «vectores normalizados → dot = coseno» es literal del notebook. El modelo que ejecuta ese código es `fuentes/modelos/modelos-2026-1.json`, fila `embed_notebook_s2` (`paraphrase-multilingual-MiniLM-L12-v2`), con `normaliza_por_defecto: false`, verificada el 2026-08-27: la API de `sentence-transformers` no normaliza sola

Cambian `normalize_embeddings` a `False` y dejan el `@` igual. ¿Qué se rompe?

- A. El código: el producto punto de vectores sin normalizar no está definido
 - B. Los puntajes dejan de ser cosenos, pero el orden de los resultados se conserva
 - C. Los puntajes dejan de ser cosenos y, además, el orden puede cambiar
- Y una segunda pregunta: ¿cómo se enteraría de esto la persona que hizo el cambio?

Cinco preguntas antes de elegir un modelo de embeddings

- 1. ¿Está entrenado con una señal de similitud? Si no, el coseno no significa relevancia
- 2. ¿Cubre el idioma del corpus? El Taller 2 elige la fila por idioma: la opción A es en español, la B en inglés
- 3. ¿Cuántos tokens acepta por fragmento? En la tabla hay un factor 64×
- 4. ¿Normaliza de fábrica? Y ojo: null no es false, es dato ausente. Se mide $\|v\|$
- 5. ¿Cuánto cuesta y dónde corre? Los de API exigen OPENAI_API_KEY, leída del entorno
- Ninguna fila gana en las cinco, y ninguna columna mide calidad: eso se mira en MTEB, con su fecha

fuentes/modelos/modelos-2026-1.json · clave embeddings · verificada 2026-08-27

modelo	dimen- siones	máx. tokens	USD por 1M tokens	multi- lingüe	normaliza por defecto
all-MiniLM-L6-v2	384	128	0	no	false
paraphrase-multilingual-MiniLM-L12-v2	384	128	0	sí, 50 idiomas	false
bge-m3	1024	8192	0	sí, 100+ idiomas	null
text-embedding-3-small	1536	8192	0,02	sí	true
text-embedding-3-large	3072	8192	0,13	sí	true

el que corre hoy en clase

128

 un cuarto de un fragmento de 512

null

 dato ausente, no es false: hay que medir $\|v\|$

Fuente: fuentes/modelos/modelos-2026-1.json, clave embeddings: cinco filas verificadas el 2026-08-27, cada una con su verified_at y su source (model cards de Hugging Face para los tres locales; documentación oficial de OpenAI para los dos de API). La pregunta 1 se responde con Reimers & Gurevych (2019), Tabla 1, p. 4 y §5, p. 7, no con la tabla. curso/talleres/taller-02-rag-vector-search.md, revisión del 2026-09-19, §«Con qué modelo corre cada parte», punto 1, y §«Parte 1», punto 3 (la fila se elige por el idioma del corpus y se declara en el informe por su id y su verified_at; líneas 62–66 y 158–162), y §«Corpus disponible» (la opción A en español, la B en inglés). curso/politica-credenciales.md: la clave se lee de variable de entorno y nunca se escribe en un notebook, una diapositiva ni una captura. La tabla verificó dimensiones, límites, precios, idioma y normalización; NO verificó calidad, así que ningún juicio comparativo de calidad se proyecta. Alamar & Grootendorst (2024), cap. 10, §«In-Depth Evaluation», p. 300 (MTEB: 8 tareas, 58 conjuntos, 112 idiomas, con tabla pública; “a good embedding model is more than just a good score on the STSB benchmark”) y §«Supervised», p. 309 (la fila embed_local_rapido de la tabla, la que la matriz del taller asigna al corpus en inglés: “performs well across many use cases and due to its small size is quite fast”)

¿La culpa es del chunking o del modelo? ¿Y qué medirías?

Fragmentos de 512 tokens, el baseline que proponen las notas del curso

Indexados con el MiniLM del notebook:
máx. 128 tokens de secuencia

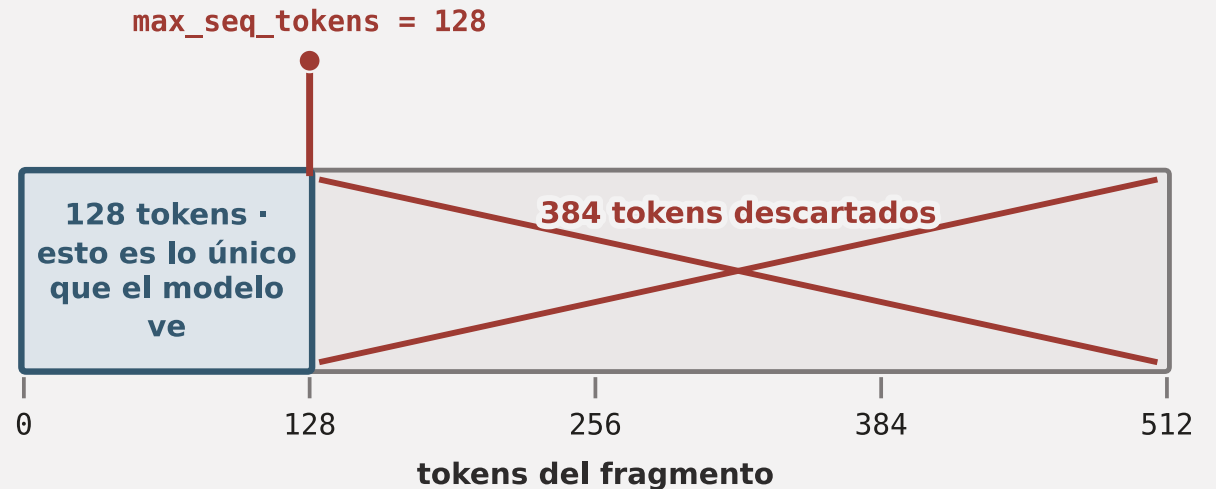
Entra el primer cuarto; se descartan 384 de cada 512, sin un solo aviso

La búsqueda no falla: devuelve resultados plausibles y en orden razonable

Ninguna consulta podrá recuperar lo que estaba en los tres cuartos finales

Entonces, ¿de quién es la culpa? ¿Y qué medirías —en una línea— para saberlo?

Fragmento de 512 tokens, techo de secuencia de 128



la búsqueda devuelve resultados, con puntajes plausibles y en orden razonable

fuentes/modelos/modelos-2026-1.json · clave embeddings · verificada 2026-08-27

Fuente: fuentes/modelos/modelos-2026-1.json, filas embed_local_rapido y embed_notebook_s2, campo max_seq_tokens: 128, verificadas el 2026-08-27, con la nota “ATENCIÓN: trunca a 128 tokens de secuencia, muy por debajo de un chunk típico de RAG”; fuentes/notas-teoricas/s2-rag-y-vector-search.md, §4.2, líneas 119–123 (chunk_size = 512, overlap = 64 como baseline); curso/talleres/taller-02-rag-vector-search.md, revisión del 2026-09-19, §«Con qué modelo corre cada parte», punto 2, y §«Parte 1», punto 3 (la fila se elige por idioma y se declara por su id y su verified_at); §«Parte 0.b» y §«Parte 1», punto 2, se leen en voz alta después de recoger las respuestas. ⚠ Las mismas notas, §4.1, línea 112, dicen “512 tokens para MiniLM”: gana el JSON por la regla de vigencia y la discrepancia está registrada y abierta

Qué NO hace un embedding

- «el gato duerme en el sofá» y «el gato no duerme en el sofá» son casi la misma frase
- Los embeddings miden de qué se habla, no qué se afirma
- No es una ley del método: es lo que le enseñaron los pares de entrenamiento a ESTE modelo
- No los uses solos donde la polaridad importa: contradicción, verificación de hechos
- Segunda debilidad: códigos, SKU y nombres propios. «error 502 nginx» buscado como «502»
- Las dos tienen respuesta y se llama búsqueda híbrida. Es el miércoles

Fuente: curso/actividades/_fuentes/s2-lun.py, líneas 80–81 (el par 2) y 96–98 (“para un modelo de embeddings, «duerme» y «no duerme» son casi la misma frase... los embeddings miden de qué se habla, no qué se afirma”, literal); curso/actividades/semana-2/s2-lun-explicacion.html, línea 119 (no usar similitud de embeddings donde la polaridad importa, sin una etapa adicional) y líneas 115–116 (el caso de «error 502 nginx»); fuentes/notas-teoricas/s2-rag-y-vector-search.md, §5.2, líneas 174–176 (la recuperación densa “puede fallar en términos técnicos exactos, IDs, números específicos”) y §5.1, líneas 166–168. Alammam & Grootendorst (2024), cap. 10, §«Embedding Models», p. 291 y Fig. 10-3, §«Generating Contrastive Examples», p. 296 y Fig. 10-8, y §«Cosine similarity», p. 303 (qué cuenta como «similar» lo fijan los pares de entrenamiento: la contradicción de NLI se usa como par negativo o se colapsa con neutral en un 0, y un modelo puede entrenarse para agrupar por polaridad). La sesión 08 es la que enseña búsqueda híbrida (curso/planificacion/plan.md y proceso/contexto/glosario.md)

Lo que se fija hoy se cobra el martes, el miércoles y el sábado

- El vector es la unidad que se indexa. Martes: HNSW e IVF sobre estos vectores
- La dimensión es memoria del índice: 3072 guarda 8× más números que 384
- El techo de tokens del modelo acota por arriba el tamaño del fragmento. Miércoles
- La métrica y la normalización son un parámetro de la colección. Martes
- El puntaje y su umbral: abstención en preguntas negativas, Parte 2 del Taller 2
- Sábado: elegir modelo de embeddings es la Parte 1.3, y hoy fue la única sesión del tema